A FOURIER EXPLANATION OF AI-MUSIC ARTIFACTS

Darius Afchar **Gabriel Meseguer-Brocal** **Kamil Akesbi** **Romain Hennequin**
Deezer Research, Paris, France
research@deezer.com

ABSTRACT

The rapid rise of generative AI has transformed music creation, with millions of users engaging in AI-generated music. Despite its popularity, concerns regarding copyright infringement, job displacement, and ethical implications have led to growing scrutiny and legal challenges. In parallel, AI-detection services have emerged, yet these systems remain largely opaque and privately controlled, mirroring the very issues they aim to address. This paper explores the fundamental properties of synthetic content and how it can be detected. Specifically, we analyze deconvolution modules commonly used in generative models and mathematically prove that their outputs exhibit systematic frequency artifacts – manifesting as small yet distinctive spectral peaks. This phenomenon, related to the well-known checkerboard artifact, is shown to be inherent to a chosen model architecture rather than a consequence of training data or model weights. We validate our theoretical findings through extensive experiments on open-source models, as well as commercial AI-music generators such as Suno and Udio. We use these insights to propose a simple and interpretable detection criterion for AI-generated music. Despite its simplicity, our method achieves detection accuracy on par with deep learning-based approaches, surpassing 99% accuracy on several scenarios.

1. INTRODUCTION

"It's not really enjoyable to make music now [...] I think the majority of people don't enjoy the majority of the time they spend making music." — M. Shulman, CEO at Suno.

Meanwhile, millions of users seem to enjoy creating AI-generated music. As a result, it was recently reported that at least a fifth of music delivered to streaming platforms is now synthetic [1]. There is undoubtedly a hype around generative AI (*GenAI*) as well as lots of investments made [2]. But this view on musical creation and enthusiasm for GenAI is far from being a majority opinion [3,4]. Lawsuits have been filed against several AI companies [5]. Beyond the largely debated ethical implications and the many social risks [6–11], AI-music has specifically raised lots of question on copyright infringements [12]. It is estimated that 24% of musicians' revenues are at risk in the next few years [13].

Most of these models are black box and owned by private companies. There is a lack of regulation to their development and use. In parallel to this rise of GenAI, AI-detection services have started to emerge [14]. Nevertheless, these services are not faring better than GenAI in that they are also mostly black box and owned by private parties. This begs the question: *What is specific about GenAI? Why do AI-detectors work? What cues may they rely on?*

This paper may be seen as an interpretability study. We take a step back from the arms race between GenAI and AI-detectors and try to ask what is specific about synthetic content and how we might detect it. We analyze deconvolution modules that are common in generative models and mathematically prove that their output will produce small frequency artifacts – *i. e.,* peaks (see Figure 3). This effect is related to the well-known *checkerboard artifact* [15]. Unlike previous empirical observations, we provide an explanation for its origin using Fourier's framework (Section 3). Interestingly, we prove these artifacts depend not on the training data nor learned weights of generative models but on their chosen architecture. In our experiments, we confirm the presence of this effect on multiple open-source models and that their structure aligns with what our theory predicts (Section 4). Further, we also show this effect happening with commercial AI-music generators (Suno and Udio). In turn, we propose a simple and interpretable criterion to detect AI-music by exploiting this phenomenon. Thanks to its underlying theory, its failure cases may be anticipated and explained. Although providing a novel detector is not our main goal but a by-product of our analysis of synthetic artifacts, our detection scores are on par with deep learning-based solutions, with over 99% accuracy.

2. RELATED WORK

2.1 AI-music detection

Mirroring the recent boom of commercial AI-music generation services (*e. g.,* Suno, Udio [16]), the task of AI-music detection is relatively novel. Only a few works have been published so far [14, 16, 17]. These early works propose CNN-like models to learn to classify real and synthetic signals. They discuss several challenges, such as the difficulty of making these detectors robust to audio manipulations. Nevertheless, a wealth of adjacent research topics may be related to this task. Voice spoofing and synthetic singing voice detection models have been proposed [18–21]. The literature on the detection of other modalities of synthetic

© D. Afchar et al.. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** D. Afchar et al., "A Fourier Explanation of AI-music Artifacts", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

media is also vast: *e. g.,* regarding video deepfakes [22], or generated text [23]. In all these works, the dominating approach is to learn to classify synthetic and real signals with black box models and then focus on their robustness. Instead, our work rather tries to explain why synthetic samples may be detected and what cues models may rely on.

2.2 Checkerboard artifact

This phenomenon was first analyzed in [15]. It was argued to be linked to overlaps of deconvolution kernels. It was also largely analyzed in computer vision for image synthesis and its detection (*e. g.,* [24–27]). In audio, this effect was first noticed by [28], which noted that it led to audible "pitched noise". In *MelGAN* [29], convolution hyperparameters were argued to be chosen properly to avoid this overlap and artifact. [30] further recommends using interpolating-upsampler to reduce the effect. Our main analysis is similar to that found in these latter works. However, while most existing work regards videos and speech, our focus on AI-music is novel and timely (*e. g.,* Suno and Udio). Then, our analysis and modeling of deconvolution artifacts in the frequency domain, as well as their usage for detection with a conveniently defined, simple linear model instead of a black box model, are not frequently found in the literature. Finally, to our knowledge, our analysis of the *architecture-dependence* and *training-* and *data-independence* had not been discussed before.

3. FOURIER ANALYSIS OF ARTIFACTS

In this work, we propose to reinterpret Convolutional Neural Networks (*CNN*) under the lenses of Fourier transforms. Instead of viewing inputs and outputs and hidden layers as time-based signals, we analyze their frequency-based decompositions (*i. e.,* spectra).

In the field of MIR, it is frequent to use spectrograms to process audio signals, as they better align with the human ear's perception. It is less common to do the same operations for everything happening within generation models. Nevertheless, this framing has many natural advantages to interpreting CNNs. For instance, a convolution in the time domain is dual to a multiplication in the frequency domain. This means a CNN may be reinterpreted as performing a series of multiplications. This property, as well as others, will be informative to better explain the emergence of generation artifacts.

Our overall proof sketch is to show that the deconvolution operation periodizes the spectra of hidden layers, hence creating peaks by tiling the constant component of the signal. Then, we explain that this property persists through the following model's operations and layers. Interestingly, our theory suggests that this phenomenon should only depend on a chosen model architecture but not on the training weight or data. We confirm this property in our next experiment section.

3.1 Background

We first report some essential concepts and properties to better understand our following arguments and proof.

Fourier transform. We denote this transform $\mathcal{F}$. For a given integrable signal $s(t)$, $\mathcal{F}[s](\xi) = \int_{-\infty}^{\infty} s(t)\ e^{-i2\pi\xi t}\,dt$. The transform is linear in s. Any multiplication (denoted $\cdot$) is transformed into a convolution (denoted $*$), and vice-versa: $\mathcal{F}[s * r] = \mathcal{F}[s] \cdot \mathcal{F}[r]$. As a reminder, a convolution is computed as $(s * r)(t) = \int_{-\infty}^{\infty} s(\tau)r(t-\tau)d\tau$. We recommend [31] for an extensive presentation of the Fourier transform.

Dirac. A *Dirac* distribution is an impulse function denoted as δ_x, with a measure $\|\delta_x\|_1 = 1$, and equal to 0 everywhere except in its parameter x. A property we will use is that a convolution with a Dirac is equivalent to applying an offset to a function: $(s * \delta_x)(t) = s(t - x)$.

Dirac comb. A *Dirac comb*, denoted as Ш_T, is defined as a sum of Dirac impulses evenly spaced with a period T: $\text{Ш}_T = \sum_{n=-\infty}^{\infty} \delta_{nT}$. The Fourier transform of a Dirac comb is also a Dirac comb, but with spacing $1/T$: $\mathcal{F}\left[\text{Ш}_T\right] = 1/T\ \text{Ш}_{1/T}$.

Dirac combs are useful to formalize the concept of *sampling*. Multiplying a continuous signal with a Dirac comb leads to a discretized version of that signal. Interpreted in the frequency domain, the Fourier transform of that discretized signal is equal to the convolution of the continuous spectrum with a Dirac comb of period $1/T$. By distributing the sum of the comb, this equals a sum of Dirac convolutions, which as we have just mentioned, results in a sum of several offsetted versions of the spectrum:

$$\mathcal{F}\left[s \cdot \text{Ш}_T\right](\xi) = \frac{1}{T} \sum_{n=-\infty}^{\infty} \mathcal{F}\left[s\right](\xi - n/T)$$

This phenomenon is called a *periodic summation*.

In Nyquist-Shanon sampling theory, this concept is used to explain the phenomenon of *aliasing*: *i. e.,* when copies of the spectrum will overlap, and why to chose the sampling frequency $1/T$ higher than twice the spectral bandwidth of a signal s to enable a perfect reconstruction of the sampled signal. In the next section, we use this phenomenon slightly differently to explain that this periodization is happening within deconvolution layers.

Deconvolution. Many generation models rely on "deconvolution" modules in their architectures. A deconvolution mirrors the way a strided convolution layer operates. Instead of shrinking an input into a latent representation, it expands it. It tiles a parametric vector (the *kernel*), which is multiplied by each coordinate of the input, this with a striding factor [32]. Deconvolutions are also sometimes called "transposed convolutions". Indeed, a strided convolution can be rewritten as a multiplication with a big matrix where the learned kernel is tiled horizontally along a diagonal with a displacement equal to the stride (Fig. 1). With this view in mind, it may be verified that the deconvolution – as defined in CNNs – corresponds to multiplying an input with a matrix with a similar but transposed structure: *i. e.,* the kernel tiled vertically with a displacement equal to the sought upsampling factor (Fig. 1).

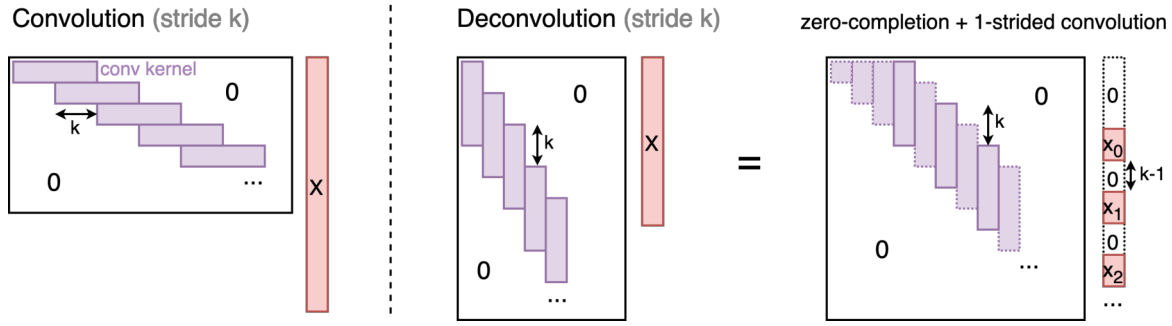


Figure 1. A CNN's convolution and deconvolution of an input X may be expressed as matrix multiplications. To build these matrices, the kernel is tiled along the diagonal with steps k (the stride). Then, it may be verified that a deconvolution is equivalent to a 1-strided convolution of the input with corresponding $k-1$ zeros inserted between each value.

Then, an important property is to realize that a strided deconvolution can be rewritten as the composition of an upsampling operation with zeros filling in-between values, followed by a 1-strided convolution. An illustration of the proof of this property is provided in Figure 1. For the complete formal proof, we refer interested readers to [32].

As reported in [15], there exist alternative definitions of deconvolution layers: an interpolated upsampling operation (*e. g.,* linear, bicubic) followed by a convolution. We will discuss this formulation later and keep the first definition of deconvolution modules for now.

3.2 Deconvolutions induce periodization

We are now ready to formulate our main analysis. As we have reported, deconvolution can be seen as performing an upsampling operation with zero-insertions between values, followed by a convolution. Let us show that this zero-insertion leads to a periodization of the signal.

Zero-upsampling is equivalent to over-sampling a discretized signal with a multiple of its sampling frequency. Let us consider a signal s, discretized with a sampling frequency f_s, from a continuous signal s^*:

$$s = s^* \cdot \text{Ш}_{1/f_s}$$

Considering a deconvolution with stride k, the zero-upsampled version v of s may be interpreted as over-sampling s with a multiple k of the frequency of f_s:

$$v = s \cdot \text{Ш}_{1/kf_s} = s^* \cdot \text{Ш}_{1/f_s} \cdot \text{Ш}_{1/kf_s} = s^* \cdot \text{Ш}_{1/f_s}$$

As consequence, from a continuous transform perspective, s and v have exactly the sample spectrum: $\mathcal{F}[s] = \mathcal{F}[v]$. However, moving to the discrete transform, while $\mathcal{F}[s]$ is read up to its proper sampling frequency f_s, $\mathcal{F}[v]$ is read up to a frequency kf_s, as shown in Figure 2. Thus, the zero-inserted signal v contains multiple clones of the spectrum of s because of the periodization effect shown above.

This is interesting since, empirically, common spectra of music signals often have most of their energy around the 0 frequency (*i. e.,* their mean value) and an exponential decrease of energy in higher frequencies. Said otherwise, music spectra look like skewed triangles [1]. Furthermore,

[1] Note: for real signals, due to the conjugate symmetry of the Fourier transform, the magnitude spectrum is mirrored for negative values of ξ.

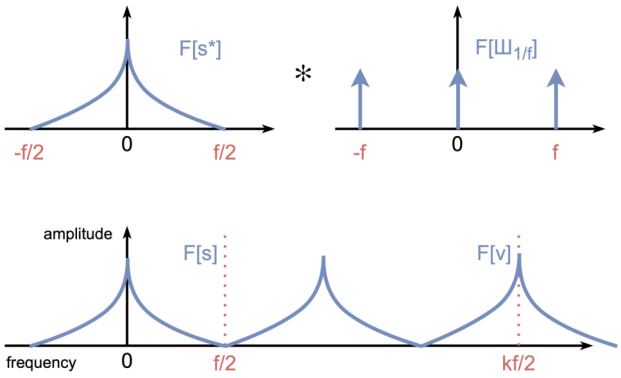


Figure 2. Periodization due to the zero-upsampling. Dotted red lines indicate the discrete signals' sampling cutoffs.

in hidden layers, it is common to add biases in the output of learnable layers, or use ReLU activations. These operations further create a bias in the time domain that leads to a "peak" at $\xi = 0$Hz in the frequency domain. Therefore, when a signal (or hidden layer vector) is periodized, this triangle pattern and its peak in 0 are cloned throughout the spectrum. We claim that this periodization of the high-energy bias is what may lead to checkerboard artifacts [15]. In music, this effect translates as a hissing noise [28, 29].

To formalize the replication, peaks in the spectrum may be found at each f_s period up to the sampling frequency $kf_s/2$ of v. Thus, let $p_{\max} = \lfloor k/2 \rfloor$, we have peaks for all integer $n \in [0\,..\,p_{max}]$ at the frequencies nf_s. For instance, a deconvolution with stride 8 leads to 5 peaks.

We have seen that the zero-upsampling of a deconvolution changes the expected distribution of the audio spectrum, from a general triangle shape, to a concatenation of triangles and peaks. How is this shape impacted by the following convolution of the layer?

As reminded, a convolution in the time domain is equivalent to a multiplication in the frequency domain. Coming back to our peaked triangles and reading this from an amplitude spectrum in log-scale: a convolution is equivalent to a vertical offset in the log-spectrum. Importantly, there is a big difference between the typical size of commonly processed music signals (*e. g.,* 1s of audio may contain 48000 samples), and the size of CNNs' kernels (*e. g.,* 3,

5 or 7). In the frequency domain, the space of possible spectra the kernel's transform lives in is parametrized by only a few parameters [2]. To multiply these two spectra, the kernel is zero-padded, which is equivalent to a *spectral interpolation* [31]. Said otherwise, the kernel spectrum is typically a slowly evolving function – relative to the variations of the spectrum to be multiplied with. While some very particular choices of kernels could mitigate the peaks (*e. g.,* a constant kernel of size the stride would cancel all peaks but the one at 0Hz), the limited expressivity due to the limited amount of parameters of the kernel is unlikely to remove all those artifacts entirely. As we will see in the next sections, this hypothesis is confirmed in practice: the output of a deconvolution actually result in creating peaks that are preserved after the convolution layer. Examples of such outputs may be found in Figure 3.

Finally, we have alluded to the fact that some variants of deconvolution may include interpolations instead of the zero-upsampling we discussed. The extension of our result to the general case of *any* possible interpolation scheme is not straightforward. Still, there is one case that can be easily extended: linear interpolation. For this, we remark that a linear interpolation may be computed as a convolution of the zero-upsampled input by a triangular filter $\Lambda(t)$ of width k, the stride of the deconvolution [3]. This convolution may be absorbed into the following convolution of the layer, and our results remain unchanged.

3.3 Effect of multiple layers

We have seen that deconvolutions, by nature of the zero-upsampling they induce, create artifacts due to the replication of the peak created by the input bias. Is this property preserved through multiple layers? It is hard to give a general answer due to the variety of architectures that exist. We discuss the case of simple, sequential, CNNs.

In this case, by induction, the periodization induced by deconvolution may lead to replicating not only the continuous component peak but also all previous peaks. This means that artifacts will be cloned in a fractal-like manner. To illustrate this property, we have computed the average spectrum found at several stages of the generator part of the Encodec model [33] in Figure 3. We can see peaks appearing at the predicted places (see the formula from the previous section), as well as preservation of previous layers' peaks in a fractal-like manner.

We formalize this recursive pattern. Instead of counting replications of the spectrum, it is convenient to count clones of the half-spectrum above 0 (the negative spectrum being symmetric for real signals). Indeed, we have seen that a spectrum from a layer i will be cloned $k^{(i+1)}/2$ times by a deconvolution $i+1$, thus creating $k^{(i+1)}$ half-spectra. By induction, a single peak at 0Hz in the input will thus lead to $P_{\max} = \prod_{i=0}^{L} k^{(i)}$ cloned half-spectra after L deconvolutions. In total, this amounts to $\lfloor P_{\max}/2 \rfloor + 1$ peaks when assembling each half-spectrum. For instance, Encodec has strides $\{8, 5, 4, 2\}$ and thus creates 161 peaks.

Interestingly, this recursive property suggests that if artifacts from previous layers are well preserved, then they may be used as a fingerprint of the whole architecture used by a generation model (specifically, the stride hyperparameter of each deconvolution).

3.4 Discussion

By combining several results from signal processing and properties of deconvolution, we have shown that generative CNNs may be subject to the generation of peaking artifacts in the spectra of their outputs. Our discussion has only regarded simple cases and does not cover all possible model architectures. For instance, we have not discussed whether non-linear activations allow these artifacts to persist, nor whether skip connections or batch normalization may help reduce the bias peak and, hence, their replication in the spectrum. In the next section, we will address these further questions empirically by analyzing the artifacts of several music generation models.

Interestingly, we did not need to consider training data or model weights to show the emergence of artifacts. This suggests that this issue seems not related to nor solvable with better training and data, but is inherent to the use of deconvolution layers in a model.

4. EXPERIMENT

In this section, we validate our theoretical findings and close the gaps left out by theory with empirical results. Our research questions are the following:

RQ1 Are the artifacts solely architecture dependent?

RQ2 Can they be used to detect synthetic music?

We show that the models we study (both open-source and closed-source) exhibit peaks in their generated outputs. In turn, as an application example, this can be used to build a simple but efficient synthetic music detector that is both fast and interpretable.

4.1 Setup

We briefly detail the models and datasets we leverage in our experiments. More implementation details may be found on our code repository [4].

Open-source models. We consider several music generation models. There are not many studies on AI-music detection. To compare our results to [17], we consider the same models: *DAC*, *Encodec* and *Musika!*, as well as the medium split of the *FMA* dataset [33–36]. The open-source nature of these models allows inspection. As argued in [17], many AI-music models rely on neural codecs (*e. g.,* [37, 38]). This means that instead of generating a

[2] which is the size of the kernel: *e. g.,* the discrete transform of a kernel K of size three is $\mathcal{F}[K](\xi) = k_0 + k_1 e^{-2i\pi\xi/N} + k_2 e^{-4i\pi\xi/N},\ \forall \xi \in [0, N-1]$

[3] With the linear interpolation, $\mathcal{F}[\Lambda](\xi) = k\,\mathrm{sinc}^2(k\xi)$. The interpolation acts as a low-pass filter. This is why this variant is less frequently used as it may lead to worse reconstructions of high frequencies.

[4] github.com/deezer/ismir25-ai-music-detector

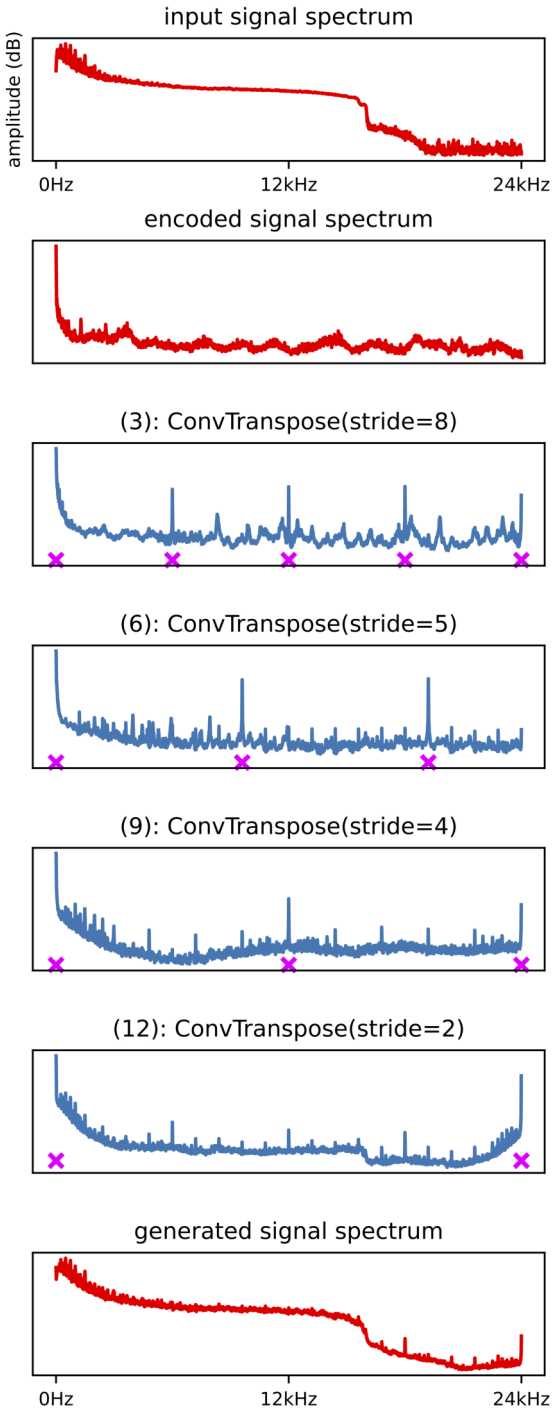


Figure 3. **Inside Encodec's decoder.** We plot the average spectra of a real music input, its latent representation (*i. e.,* after the dequantization of the audio tokens), as well as all its successive hidden outputs of all the deconvolution layers of Encodec (48kHz version). We indicate the layer number and chosen stride hyper-parameter of each deconvolution. We plot crosses below the signal where the periodized continuous component might create an artifact due to the periodization of each above spectrum (as deduced from the stride). The spectra are averages from the many channels of each hidden layer, as well as temporally averaged from successive 2^{13} samples.

song end-to-end, it is more efficient to learn to generate a sequence of audio tokens, that are then dequantized and converted to 48kHz audio through the neural codec decoder. It was shown that learning to detect neural codec then transfers to the detection of music generators using that codec [17]. We will compare our results to the black box detection model proposed in that latter study. To address *RQ1*, we also consider the *MTAT* and *MTG-Jamendo* datasets [39, 40].

Closed-source models. We also conduct experiments on *Suno* and *Udio*. We leverage the recent *SONICS* dataset [16], containing 50k of such synthetic music tracks. With these models, we cannot verify whether the presence of peaks corresponds to the underlying architecture. We only showcase that these models exhibit the same issues.

Artifact fingerprint. To study the presence of artifacts, we need to extract the potential replicated peaks in generated spectrograms. With the analysis provided in Section 3, it seems relevant to start from average spectra, as proposed in Figure 3, and related to what was proposed in computer vision in [27]. Indeed, while a local music patch will exhibit many frequencies due to its melodic content, it is reasonable to think that when averaging many patches' spectrum over several minutes of audio, the melodic content will be smoothed and result in the triangle-like shape we were discussing. From there, we simply propose to subtract the local minima of the spectrum over sliding windows to highlight the local variations we try to detect (*i. e.,* peaks). Finally, we consider a reduced bandwidth (*e. g.,* [5kHz, 16kHz]) to further discard melodic information and uninformative noise above the cutoff of the mp3 codec. We refer to this small processing of the spectrum as computing an *artifact fingerprint*. It represents the local variations in the amplitude of the average spectrum (*e. g.,* see Figure 4).

4.2 Architecture dependence

To address *RQ1* that the studied artifacts are independent of training data and learned weights, we train several models of *DAC* [34], using the same model configuration as *VampNet* [37]. We train DAC on the FMA twice, with a different random seed (impacting the weight initialization and optimization), on MTAT and MTG-Jamendo. We compute artifact fingerprints of the auto-encoded tracks (following the same methodology as [17]). The averages of the found artifact fingerprints (on the test set) are displayed in Figure 4. Strikingly, we find that the placement of peaks is the same for all four models. Their amplitudes are slightly different between the four versions but still clearly exhibit the same overall pattern. This means that artifacts are indeed: 1) *Weights-independent*, since different training seeds do not change the peaks on FMA; 2) *Data-independent*, since we find the same peaks for models trained on MTAT and Jamendo. Note that tracks from MTAT have a sampling rate of 16kHz, hence the cutoff that we observe at 8kHz. Overall, this suggests that AI-music artifacts are indeed solely *architecture dependent*.

This also suggests that a detector trained on one given

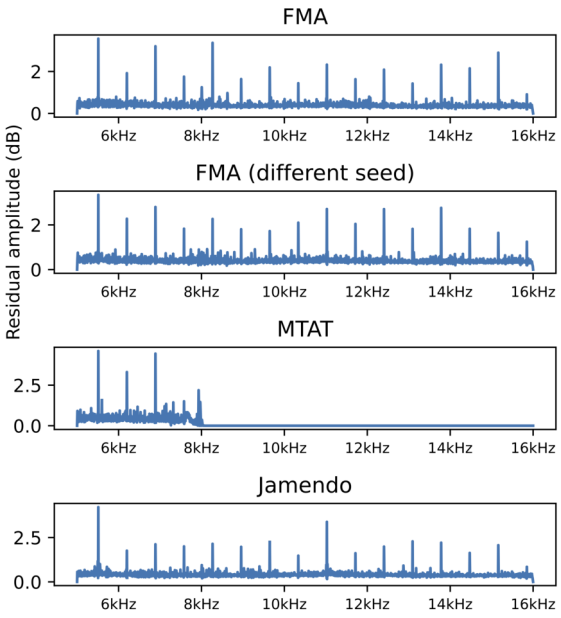


Figure 4. Artifacts of various trained versions of DAC.

model and relying on these cues may transfer its performance to that model trained on a different dataset, or even other models that share a similar parametrization of their deconvolution layers. This may explain the intra-family generalization property found in [17].

4.3 AI-music detection

To address *RQ2*, we build a simple detector. From our previous section, it seems that synthetic content may be simply detected by checking if the artifact fingerprint contains peaks or not. By the look of what was found in Figure 4, it seems that a straightforward linear regressor could be enough for this task: *e. g.,* learning positive coefficients for peaking frequencies and negative ones for baseline values.

We first follow the setup of [17]. We train to detect each real music track in FMA against its synthetic reconstructions. The results are provided in Table 1. We use the neural codecs at their maximum bandwidth, which is the hardest to detect. Overall, the performances are comparable between our linear model and the reported black box CNN model, and the classification is almost perfect.

Class	**Our**	*Reported from [17]*
Real	99.87	99.7
Synthetic		
↪ DAC (14kbps)	99.68	99.3
↪ Encodec (24kbps)	99.81	99.7
↪ Musika!	99.97	100.0

Table 1. Test detection scores (%) on open-source models. We report the synthetic class breakdown.

Next, we do the same experiment with closed-source generators. We follow the setup of SONICS [16]. However, the real audio tracks were not provided due to copyrights. Therefore, we fall back to using tracks from FMA instead. We resampled them to 16kHz as [16]. This means our transforms have a cutoff at 8kHz. We accordingly adjust the bandwidth of our fingerprints to [1kHz, 8kHz]. This change, unfortunately, means that we cannot compare our results in a fair manner. We nevertheless report SONICS' best model (*SpecTTTra-α*) in Table 2 for information. As it may be seen, the scores of our 10K-parameter logistic regression are comparable to the ones of a 20M-parameter transformer model. On the synthetic version seen during training (*Suno v3.5* and *Udio 130*), the classification is perfect. The performance drops on *Udio 32*, which was unseen during training (following the splits from [16]). We suspect that Udio might have changed their model architecture between the 32 and 130 versions, which would explain the failure at the zero-shot performance transfer.

Class	Our	*Reported from [16]*
Real	99.97	99
Synthetic		
↪ Suno v3.5	100.00	100
↪ *Suno v3*†	100.00	96
↪ *Suno v2*†	99.90	78
↪ Udio 130	100.00	100
↪ *Udio 32*†	39.83	96

Table 2. Test detection scores (%) on closed-source models. We include the breakdown for Suno and Udio. † indicate versions unseen during training.

We may also train our model to detect each generator individually and display the learned weights as a way to highlight the found peaks (see Figure 5). We can see clear patterns and various peak placements from DAC, Encodec, and Suno. This is more fuzzy for other models that are yet still well detected with our method. We leave these more challenging interpretations for future work.

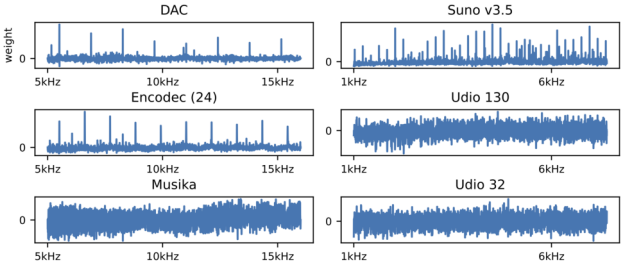


Figure 5. Learned logistic regression coefficients.

5. CONCLUSION

We propose a theoretical analysis for the emergence of peak artifacts in AI-music, formalize their recursive shape, and predict their architecture independence. Our experiments confirmed this latter observation, and we have used our new-found knowledge to craft a simple detector with performances on par with previous million-parameter models. While this highlights what cues detectors may rely on, other types of artifacts might still exist: *e. g.,* this may explain how *SpecTTTra* generalizes to the unseen *Udio 32*. We also have not touched on the topic of audio manipulation robustness. We can already anticipate that those impacting frequency positions will affect our model (*e. g.,* resampling, pitch shift). We leave this as future work.

6. REFERENCES

[1] Deezer, "Deezer deploys cutting-edge AI detection tool for music streaming," https://newsroom-deezer.com/2025/04/deezer-reveals-18-of-all-new-music-uploaded-to-streaming-is-fully-ai-generated/, 2025, [Online; accessed 22-March-2025].

[2] D. G. Widder and M. Hicks, "Watching the generative AI hype bubble deflate," *arXiv preprint arXiv:2408.08778*, 2024.

[3] T. Sippy, F. Enock, J. Bright, and H. Z. Margetts, "Behind the Deepfake: 8% Create; 90% Concerned. Surveying public exposure to and perceptions of deepfakes in the UK," *arXiv preprint arXiv:2407.05529*, 2024.

[4] 404 Media, "CEO of AI Music Company Says People Don't Like Making Music," https://www.404media.co/ceo-of-ai-music-company-says-people-dont-like-making-music/, 2025, [Online; accessed 13-June-2025].

[5] The Guardian, "Music labels sue AI song generators Suno and Udio for copyright infringement," https://www.theguardian.com/music/article/2024/jun/25/record-labels-sue-ai-song-generator-apps-copyright-infringement-lawsuit, 2025, [Online; accessed 13-June-2025].

[6] L. Pelly, *Mood Machine: The Rise of Spotify and the Costs of the Perfect Playlist*. Hodder & Stoughton, 2025.

[7] Y. Wei, Y. Zhu, P. Hui, and G. Tyson, "Exploring the Use of Abusive Generative AI Models on Civitai," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 6949–6958.

[8] H. H. Jiang, L. Brown, J. Cheng, M. Khan, A. Gupta, D. Workman, A. Hanna, J. Flowers, and T. Gebru, "AI Art and its Impact on Artists," in *AIES*. ACM, 2023.

[9] S. Gautam, P. N. Venkit, and S. Ghosh, "From Melting Pots to Misrepresentations: Exploring Harms in Generative AI," in *GenAICHI*, 2024.

[10] M. Klincewicz, M. Alfano, and A. E. Fard, "Slopaganda: The interaction between propaganda and generative AI," *Filosofiska Notiser*, vol. 12, no. 1, pp. 135–162, 2025.

[11] L. Klein, M. Martin, A. Brock, M. Antoniak, M. Walsh, J. M. Johnson, L. Tilton, and D. Mimno, "Provocations from the Humanities for Generative AI Research," *arXiv preprint arXiv:2502.19190*, 2025.

[12] T. S. Goetze, "AI Art is Theft: Labour, Extraction, and Exploitation: Or, On the Dangers of Stochastic Pollocks," in *ACM FAccT*, 2024.

[13] CISAC, "Study on the economic impact of Generative AI in the Music and Audiovisual industries," https://www.cisac.org/Newsroom/news-releases/global-economic-study-shows-human-creators-future-risk-generative-ai, 2024, [Online; accessed 22-March-2025].

[14] Y. Li, M. Milling, L. Specia, and B. W. Schuller, "From Audio Deepfake Detection to AI-Generated Music Detection–A Pathway and Overview," *arXiv preprint arXiv:2412.00571*, 2024.

[15] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and Checkerboard Artifacts," *Distill*, 2016. [Online]. Available: http://distill.pub/2016/deconv-checkerboard

[16] M. A. Rahman, Z. I. A. Hakim, N. H. Sarker, B. Paul, and S. A. Fattah, "SONICS: Synthetic Or Not - Identifying Counterfeit Songs," in *International Conference on Learning Representations (ICLR)*, 2025.

[17] D. Afchar, G. Meseguer-Brocal, and R. Hennequin, "AI-Generated Music Detection and its Challenges," in *ICASSP*. IEEE, 2025.

[18] Z. Wu, J. Yamagishi, T. Kinnunen, C. Hanilçi, M. Sahidullah, A. Sizov, N. Evans, M. Todisco, and H. Delgado, "ASVspoof: the automatic speaker verification spoofing and countermeasures challenge," *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, no. 4, 2017.

[19] Z. Almutairi and H. Elgibreen, "A review of modern audio deepfake detection methods: challenges and future directions," *Algorithms*, vol. 15, no. 5, p. 155, 2022.

[20] C. Sun, S. Jia, S. Hou, and S. Lyu, "AI-synthesized voice detection using neural vocoder artifacts," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 904–912.

[21] Y. Zang, Y. Zhang, M. Heydari, and Z. Duan, "Singfake: Singing voice deepfake detection," in *ICASSP*. IEEE, 2024.

[22] Y. Mirsky and W. Lee, "The creation and detection of deepfakes: A survey," *ACM computing surveys (CSUR)*, vol. 54, no. 1, 2021.

[23] L. Lin, N. Gupta, Y. Zhang, H. Ren, C.-H. Liu, F. Ding, X. Wang, X. Li, L. Verdoliva, and S. Hu, "Detecting Multimedia Generated by Large AI Models: A Survey," *arXiv:2402.00045*, 2024.

[24] X. Zhang, S. Karaman, and S.-F. Chang, "Detecting and simulating artifacts in gan fake images," in *2019 IEEE international workshop on information forensics and security (WIFS)*. IEEE, 2019, pp. 1–6.

[25] T. Osakabe, M. Tanaka, Y. Kinoshita, and H. Kiya, "CycleGAN without checkerboard artifacts for counter-forensics of fake-image detection," in *International Workshop on Advanced Imaging Technology (IWAIT) 2021*, vol. 11766. SPIE, 2021, pp. 51–55.

[26] B. Liu, F. Yang, X. Bi, B. Xiao, W. Li, and X. Gao, "Detecting generated images by real images," in *European Conference on Computer Vision*. Springer, 2022, pp. 95–110.

[27] R. Corvi, D. Cozzolino, G. Poggi, K. Nagano, and L. Verdoliva, "Intriguing properties of synthetic images: from generative adversarial networks to diffusion models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 973–982.

[28] C. Donahue, J. McAuley, and M. Puckette, "Adversarial audio synthesis," in *International Conference on Learning Representations*, 2018.

[29] K. Kumar, R. Kumar, T. De Boissiere, L. Gestin, W. Z. Teoh, J. Sotelo, A. De Brebisson, Y. Bengio, and A. C. Courville, "Melgan: Generative adversarial networks for conditional waveform synthesis," *Advances in neural information processing systems*, vol. 32, 2019.

[30] J. Pons, S. Pascual, G. Cengarle, and J. Serrà, "Upsampling artifacts in neural audio synthesis," in *ICASSP*. IEEE, 2021.

[31] S. Mallat, *A Wavelet Tour of Signal Processing, Third Edition: The Sparse Way*, 3rd ed. USA: Academic Press, Inc., 2008.

[32] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," *arXiv preprint arXiv:1603.07285*, 2016.

[33] A. D'efossez, J. Copet, G. Synnaeve, and Y. Adi, "High Fidelity Neural Audio Compression," *ArXiv*, vol. abs/2210.13438, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:253097788

[34] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-Fidelity Audio Compression with Improved RVQGAN," *NeurIPS*, vol. 36, 2024.

[35] M. Pasini and J. Schlüter, "Musika! Fast Infinite Waveform Music Generation," in *ISMIR*, 2022.

[36] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *ISMIR*, 2017. [Online]. Available: https://arxiv.org/abs/1612.01840

[37] H. F. Garcia, P. Seetharaman, R. Kumar, and B. Pardo, "VampNet: Music Generation via Masked Acoustic Token Modeling," *ISMIR*, 2023.

[38] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and Controllable Music Generation," *NeurIPS*, vol. 36, 2024.

[39] E. Law, K. West, M. I. Mandel, M. Bay, and J. S. Downie, "Evaluation of Algorithms Using Games: The Case of Music Tagging," in *ISMIR*. Citeseer, 2009, pp. 387–392.

[40] D. Bogdanov, M. Won, P. Tovstogan, A. Porter, and X. Serra, "The MTG-Jamendo Dataset for Automatic Music Tagging," in *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML 2019)*, Long Beach, CA, United States, 2019. [Online]. Available: http://hdl.handle.net/10230/42015